

Phase 2.1 Report Document

Horizontal Design Discussion

In Phase 1, we originally rapidly designed the app on paper to have a summarized home screen with important reminders at the top, buttons with hotlinks to other pages, and the bottom portion of the page to be a quick overview of current goals and spending visualized with graphs. In addition to the homescreen, we had two other pages: a transaction page and a goals and spending reports page. From that, we created a photorealistic mockup as well as a low to medium fidelity prototype that previewed our homescreen and other pages. From the feedback we received from our instructors, we decided to make the following changes to our current prototype:

1. In our previous prototypes, the space was not utilized well with some features being too small and difficult to see as well as some visual elements being laid out in a way where it is difficult to group visualizations and features together. In our current prototype we decided to implement these following changes:
 - a. For any data entry form, we decided to create a grid layout where the numeric fields (e.g., date, amount of money) that logically wouldn't need to take up a lot of space were significantly smaller than the longer text fields (e.g., title, category). We then placed smaller numeric fields either inline together or next to those larger text fields.
 - b. Visualizations were kept on their own respective pages and made to look distinct from each other so that the user could easily associate a unique visualization to a specific page instead of potentially confusing visualizations across multiple pages (i.e., associating pie charts with the transactions and goal pages instead of just the transactions page)
 - c. In order to conserve space, we decided to implement sequencing and create conditional fields and popups so that if the user does need to input any additional information because of their selection, it's

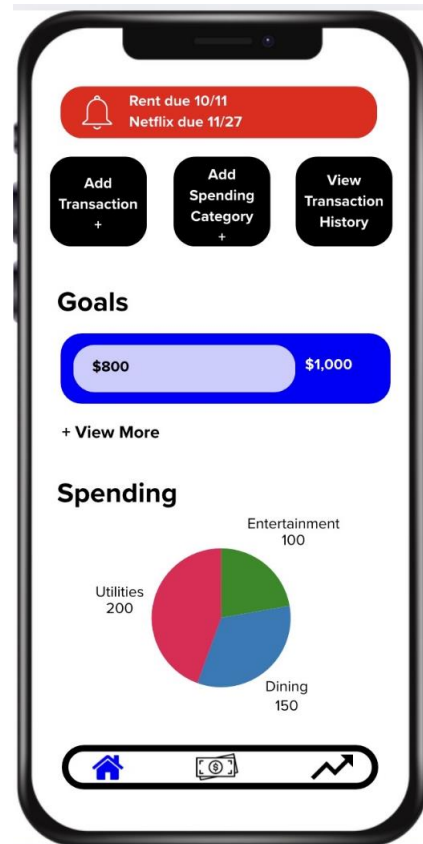
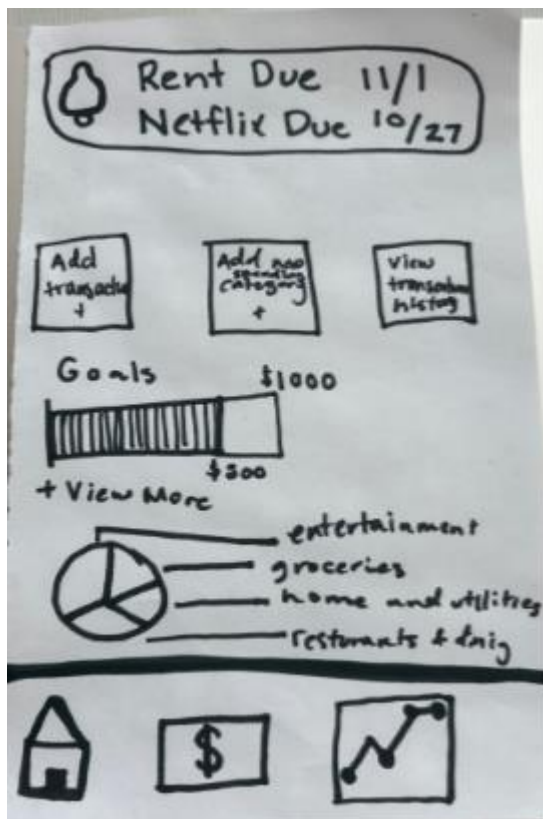
requested and if it's not needed it will not take up unnecessary space on the page.

- d. We also decided to group specific tasks within the page inside cards or widgets, implementing the Gestalt Principle of proximity. We used the relationship between screen elements to group them in cards therefore disassociating the other unrelated elements from them.
2. Across our previous pages, we did not adhere to some inclusive design guidelines. Our first color scheme did not have a high enough color contrast ratio between the background and foreground colors and we did not choose an accessible and readable font. We decided in our current design to change our main background color to #0000FF and if another accent color is needed #CDCDFF is used along with #FFFFFF and #000000 for text colors. In regards to font, we decided to pivot to the sans serif font "Proxima Nova", which is widely considered to be an accessible font.
3. Our original home page should have not been a generic home page, it should have a distinct feature and specific use. This page should have been a landing page, acting as a starting point for users when they launch the software. We decided to change our home page summary style and expand on the reminders feature more by creating a full deadlines and to-do page so that users can easily see what payments and tasks they have to complete. Our team felt like this feature would be the most appropriate as a starting point for our users as this feature contains time-sensitive and important information that users should be able to see at a glance.

CMSC 434

Melanie, Aditi, Sammi, Aidana

11/7/25



Reminders & To-Do List

Reminders

Rent due
Oct 30, 2025
Credit card payment
Oct 25, 2025
Car insurance renewal
Nov 1, 2025

Reminder title

mm/dd/yyyy

Add Reminder

To-Do List

☐ Update monthly budget
☐ Pay utilities bill

Home Transactions Goals

Deadlines & To-Do

Deadlines

Add a Deadline

Earliest first

Add

Title (e.g., Rent due)

mm/dd/yyyy

Add notes (optional)

No deadlines yet

Deadlines Income Transactions Goals

4. In previous prototypes, the navigation bar should have included text labels for clarity. We originally assumed the symbols would be easy metaphors for the user to understand with no need for text. We quickly realized from the provided feedback that these symbols could be ambiguous and have multiple meanings leaving the pages unintuitive and vague. In our current prototype, we included text labels in the navigation bar so that users would know exactly what each page does.
5. Finally, our prototypes originally did not follow some specification requirements listed in the project description. For example, our screen size was not the specified dimensions, our font size was too small, and our low to medium fidelity prototype required scrolling. Our team quickly fixed our current prototype to fit these development requirements by making a border around the screen (creating a visual and physical barrier), increasing font size, and minimizing the need for scrolling by condensing information and if needed allowed the ability to scroll internally within the app but not within the browser.

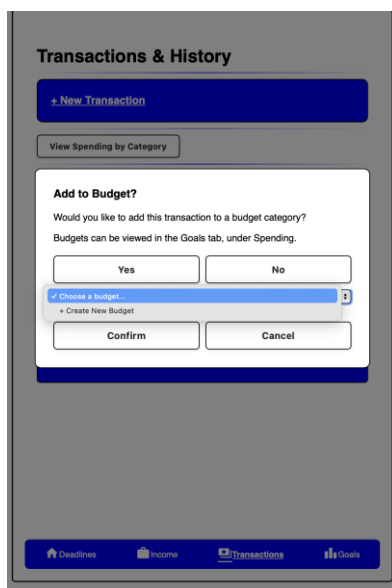
Vertical Design Discussion

Our core tasks were based on the task scenarios we had developed in Phase 1, and were later refined through feedback and the additional task scenarios posted in the Phase 2 Description. We were able to implement a few instances of vertical design within our Phase 2.1 implementation.

One of our vertical design implementations was our transaction entry and storage. In order to enter and store new transactions, the user first has to click on the “+ New Transaction” at the top of the transactions page. Upon clicking, it will expand into a slightly larger form, allowing the user to enter details regarding their transaction, including name, date, amount, and the category they wish to sort it under. The user is presented with four “pre-created” categories (Food, Utilities, Entertainment, Home) and is able to make their own custom user-created categories as per their needs. Once they enter the necessary information, they can click the “Add Transaction” button and it will automatically be added into the Transaction History section at the bottom of the page. If the user decides they actually do not want to add the transaction to their history, they can click the “Cancel Transaction” button at any time to reset.

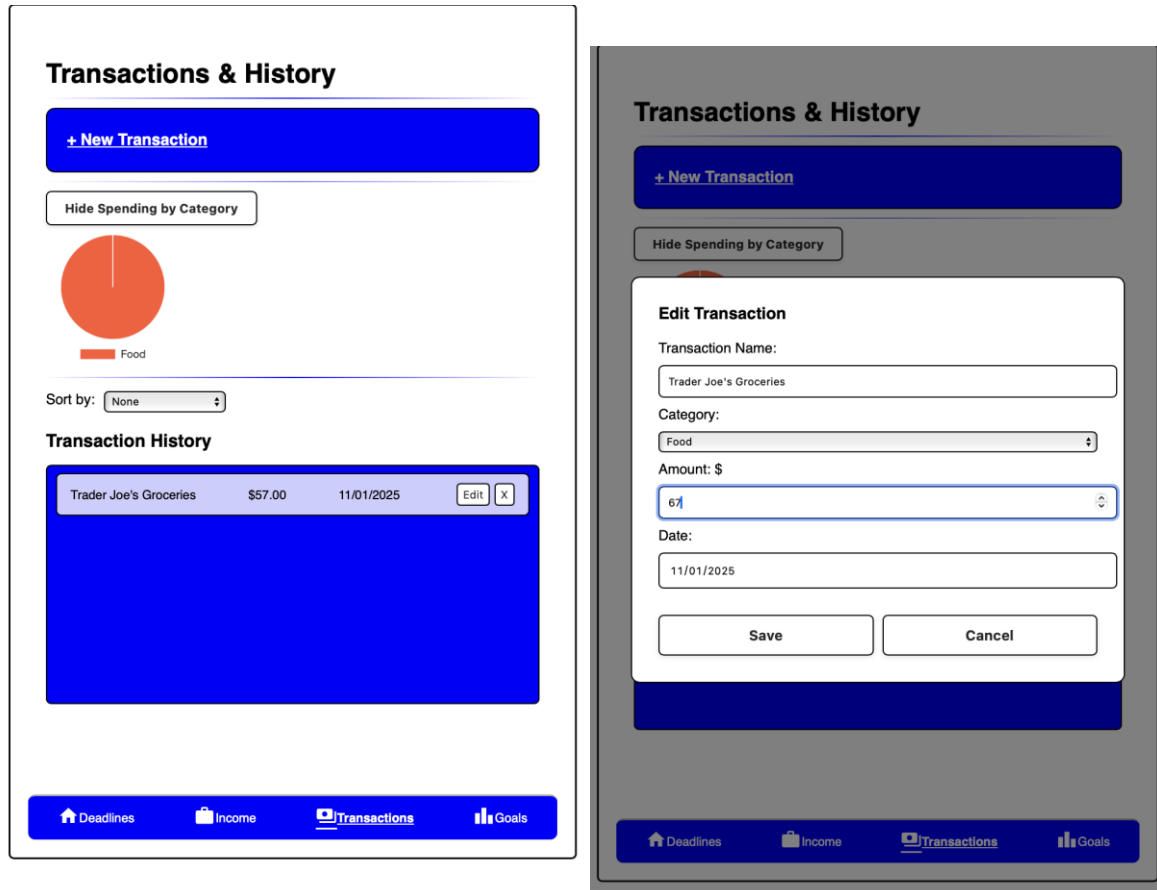
The screenshot shows a mobile application interface for managing transactions. At the top, there's a header "Transactions & History". Below it is a blue button labeled "+ New Transaction". Clicking this button expands a form. The form has three input fields: "Name:" with the text "Trader Joe's Groceries", "Amount: \$" with the value "\$7", and "Date:" with the date "11/01/2025". Below these is a "Category:" section with a dropdown menu showing "Food" and a button "+ Add New Category". At the bottom of the form are two buttons: "Add Transaction" and "Cancel Transaction". Below the form is a button "View Spending by Category". Underneath that is a "Sort by:" dropdown menu currently set to "None". Below the sorting options is a section titled "Transaction History" which contains a large blue rectangular area, likely a placeholder for a list of transactions. At the very bottom of the screen is a navigation bar with four icons and labels: "Deadlines", "Income", "Transactions" (which is highlighted), and "Goals".

Each time a user adds a new transaction, they are also given the option to add that transaction to a Budget Category. Budgets are listed underneath the Goals page, in the spending section. This allows the user to make sure they are staying under budget for certain categories of transactions, if they wish to set certain limits. If their transaction does not fit into one of their pre-existing budgets, they can also create custom budgets, which will be reflected in the Goals page, in the spending section.



As per the task scenarios, if a user wishes to see a visual breakdown of their spending/transactions, they can click the “View Spending By Category” button which will automatically produce a pie chart, color coded based on the breakdown of the total amount in each of the categories. If a user wishes to see the list of transactions instead of a visual representation, they can view the Transaction History section at the bottom, where each transaction they inputted will be listed. Each transaction is editable and removable. If a user realizes they made a mistake while entering the transaction, they are able to click the “Edit” button, which triggers a pop-up, allowing them to make edits to the transaction. The “X” button triggers a pop-up asking them to confirm whether they actually meant to delete the

transaction, avoiding any accidental mishaps.



Another one of our vertical design implementations can be seen with the entry of savings goals and budget limits on the “Goals” page. As mentioned before, users are prompted to add to the goals/limits on this page in the Income/Transactions pages respectively. However, they can also manually add goals/limits on the Goals page itself. The landing page is the “Savings” side, where the user is prompted to enter the name of the goal and the target amount, pressing the “Add” button on the right to save it into the ledger. Users can also update existing goals by entering in amounts with the record deposit functionality and pressing the “Record” button. The progress bar and percentage associated will update accordingly, reflecting a green color instead of the default blue/gray color once the goal has been reached/exceeded. If the user wants to see the individual deposits that make up a

specific goal, they can do so by clicking on the dropdown arrow to the right of each goal. The total saving across all of the goals is also reflected in the value on the top right.

To flip to the “Spending” side of the Goals tab, the user is prompted to click on the “View Spending →” button near the bottom of the page.

The “Spending” side of the Goals tab has much of the same functionality, just adjusted to fit spending limits and budget categories. Again, the user is prompted to enter in the name of the budget and the target limit, pressing “Add” to save it into the ledger. Users can update existing budgets in the same way as the “Savings” tab, and the progress bar will reflect red instead of the default blue/gray color once the limit has been reached. Again, if the user wants to see the individual expenses that make up a goal, they can do so by clicking on the dropdown arrow. The total spending across all budgets is reflected in the value on the top right.

Savings 1523.00

Goals Track progress per goal

Goal name (e.g., Vacation) Target Add

Vacation to Bali
523.00 of 2000.00 (26%)

View Deposits (2)

Record deposit Record

Tuition
1000.00 of 1000.00 (100%)

View Deposits (1)

Manual Entry 2025-11-07 +1000.00

Record deposit Record

View Spending →

Spending 260.00

Budgets Record spending per budget

Budget name (e.g., Food) Limit Add

Take Out Food
40.00 of 400.00 (10%)

View Expenses (2)

Manual Entry 2025-11-07 30.00

Manual Entry 2025-11-07 10.00

Record expense Record

Shopping
220.00 of 200.00 (110%)

View Expenses (2)

Record expense Record

← Back to Savings

Deadlines Income Transactions Goals

A third vertical design implementation was our income entry and storage. In order to add in new income streams the user would enter in the title or name of their income and select what type of income they earned. Once selected, information fields related to the type of income would appear. If the user selects “Hourly” the fields “Hourly Rate”, “Hours Worked”, and “Date” would appear and would be required to save the income. Similarly, if the user selects “Lump Sum” the fields “Amount”, “Date”, “Notes” would appear—only the date and amount earned would be required and the notes field would be optional for user convenience. Finally, if the user selects “Annual Salary” the fields “Salary”, “Pay Frequency”, and “Date” would appear which would all be required.

The image displays three vertical wireframes of an income entry form, illustrating different states of the user interface. Each wireframe features a blue header with the title "Income" and a green subtitle "Total Income 2719.77".

Wireframe 1 (Left): Shows the initial form state. It includes a "+ Add Income" button, a "View Ledger" button, and a text input field containing "barista". Below this is the "Type of Income" section with three radio buttons: "Hourly" (selected), "Lump sum (odd jobs / one-off)", and "Annual salary". The "Hourly details" section contains three input fields: "Hourly rate" (with "e.g., 18.50"), "Hours Worked" (with "e.g., 20"), and "Date" (with "mm/dd/yyyy" and a calendar icon). At the bottom are "Save income" and "Reset" buttons.

Wireframe 2 (Middle): Shows the form after the "Lump sum" option is selected. The "Type of Income" section now has "Lump sum (odd jobs / one-off)" selected. The "Lump sum details" section is expanded, showing "Amount" (with "e.g., 250.00"), "Date" (with "mm/dd/yyyy" and a calendar icon), and "Notes (optional)" (with "e.g., Moved furniture for neighbor; paid via cash app.").

Wireframe 3 (Right): Shows the form with the "Lump sum details" section expanded, similar to the middle wireframe, but with the "Notes (optional)" field containing the example text "e.g., Moved furniture for neighbor; paid via cash app.".

All three wireframes share a common bottom navigation bar with four icons: a house for "Deadlines", a bar chart for "Income", a document for "Transactions", and a target for "Goals".

Once all the required fields are filled out the user can select “Save Income” to save the income information or “Reset” to clear the form. Each time the user enters new

income, they will be asked if they would like to add a portion of their income into their savings. If yes, the user will then be prompted to enter how much they would like to save and what savings goal they would like to attribute it to. The maximum amount of money to be saved is calculated by their income type and attributes. For example, if the user entered an hourly income, then the maximum savings cap would be their hourly rate multiplied by their hours worked. Similarly, the maximum savings for an annual salary would be the salary amount divided by payment frequency in weeks. If the user entered a lump sum amount there would be no calculations done and their maximum amount would be equal to the amount that they originally entered. To choose a saving goal, the user would have to select one from the drop down menu or select the option “+Create New Goal” which would create a new goal visible on the “Goals” page. Once all the information is entered the user can click “Confirm” to add to their savings goal or “Cancel” to reset the form.

The image displays two side-by-side screenshots of a mobile application interface, likely for financial management. Both screens show the 'Income' section with a 'Total Income' of 280.00. The left screen shows the 'Add Income' form with fields for 'Title / Name of Income' (e.g., Campus Barista, Freelance, Company Name) and 'Type of Income' (Hourly, Lump sum (odd jobs / one-off), Annual salary). The right screen shows the same form with the 'Add to Savings?' modal open. The modal asks 'Would you like to add a portion of this income to a savings goal?' and provides 'Yes' and 'No' buttons. Below the buttons, it shows 'Max: \$280.00' and a 'Select Savings Goal:' dropdown menu with the option 'Choose a goal...'. At the bottom of the modal are 'Confirm' and 'Cancel' buttons. The bottom navigation bar of both screens includes icons for 'Deadlines', 'Income', 'Transactions', and 'Goals'.

Income
Total Income 280.00

+ Add Income
View Ledger

Title / Name of Income
e.g., Campus Barista, Freelance, Company Name

Type of Income
☐ Hourly
☐ Lump sum (odd jobs / one-off)
☐ Annual salary

Add to Savings?
Would you like to add a portion of this income to a savings goal?
Yes No

Max: \$280.00

Select Savings Goal:
Choose a goal...

Confirm Cancel

Deadlines Income Transactions Goals

At any time, the user can view their total income at the top of the page or click the “View Ledger” button to view a pop up summary table of their income history. The ledger will contain the date, title/name, type of income, details (e.g., visual calculation or notes), and amount earned. While in the ledger popup, the user can choose to remove entries by pressing the “Remove” button next to that specific entry. If the user would like to clear the entire table they can select the “Clear All” button. Once selected, the user will be asked to confirm their decision and will be notified that it will permanently delete every income from the ledger. This is a safeguard so that if a user accidentally pressed “Clear All” they can press “Cancel”. If not, they will confirm their decision with the “Delete All” button. If income is added or removed the heading labeled “Total Income” in green will update with a running total.

